

Project Workflow: Week 10

Project: EasyStego: A Drag-and-Drop Image Steganography Tool

Phase: Final Testing, Documentation & Submission

Submitted by: Md. Salauddin | ID: 11240321728

Course: Software Development I (CSE 2216)

Submitted to: Vashkar Kar, Lecturer, Department of CSE, NUBT Khulna

1. Objective

The primary goal of Week 10 was to complete full end-to-end system testing of the EasyStego application, finalize all project documentation, capture screenshots, and submit the completed project. This included:

- Performing complete system testing of both the Hide Message and Reveal Message workflows.
- Testing edge cases including oversized messages, unsupported file formats, and empty inputs.
- Finalizing project documentation including the project report and README.md.
- Capturing screenshots of the running application for documentation purposes.
- Uploading the final EasyStego.java to GitHub and submitting all project documents.

2. Detailed Task Breakdown

Full System Testing — Hide Workflow: Tested the complete Hide Message workflow end-to-end. Dropped PNG images of varying dimensions (500x500 to 3000x2000 pixels) into the Hide tab, typed messages of different lengths, clicked Encrypt & Save, and verified that the output PNG file is saved correctly. Confirmed output images are visually identical to originals using side-by-side comparison.

Full System Testing — Reveal Workflow: Tested the complete Reveal Message workflow end-to-end. Dropped each saved stego-image into the Reveal tab and confirmed the exact original message is recovered without any data loss or character corruption. Verified the Copy to Clipboard button copies the decoded text correctly.

Edge Case Testing: Tested and verified the following edge cases: (1) clicking Encrypt without loading an image — error dialog fires correctly; (2) clicking Encrypt with an empty message — error dialog fires correctly; (3) dropping a non-image file — `importData()` returns false gracefully; (4) attempting to embed a message that exceeds image capacity — confirmed by capacity label and verified behavior; (5) dropping a JPEG image — confirmed it is converted and saved as PNG losslessly.

Final Documentation: Reviewed and finalized the project report covering all sections: Abstract, Introduction, Literature Review, Methodology, Implementation, Results and Analysis, Discussion, Conclusion, and References. Verified all method names, class names, and pseudocode match the final submitted source code.

Screenshots: Captured two application screenshots: (1) the Hide Message tab with a loaded image, capacity display, and typed message; (2) the Reveal Message tab showing a successfully decoded hidden message. Screenshots were added to the project documentation.

GitHub Final Upload: Uploaded the final EasyStego.java to the GitHub repository. Verified all weekly uploads are present in the commit history. Confirmed the README.md renders correctly on the repository home page.

Project Submission: Compiled and organized all submission documents — project proposal, 10-week work plan, weekly workflow reports (Weeks 1 to 10), requirements document, system workflow diagram, UI wireframe sketches, project report, and final source code — and submitted them as required.

3. Test Results Summary

All test cases passed successfully. Results are summarized below:

#	Test Case	Expected	Result
1	Embed message into 500x500 PNG and recover it	Message recovered without data loss	✓ Pass
2	Embed message into 3000x2000 PNG and recover it	Message recovered without data loss	✓ Pass
3	Output PNG visually identical to original image	No visible difference to the naked eye	✓ Pass
4	Encrypt button clicked with no image loaded	Error dialog displayed	✓ Pass
5	Encrypt button clicked with empty message	Error dialog displayed	✓ Pass
6	Non-image file dropped into drop zone	Drop rejected gracefully, no crash	✓ Pass
7	JPEG image dropped — saved as PNG losslessly	Output file saved as PNG, message recoverable	✓ Pass
8	Copy to Clipboard button copies decoded message	Text copied to system clipboard correctly	✓ Pass
9	Reveal tab on image with no hidden message	Fallback message displayed, no crash	✓ Pass
10	Application runs on Windows with JDK 11+	Window launches and all features functional	✓ Pass

4. Final Project Summary

The following table summarizes what was built across all 10 weeks:

Week	Phase	Key Output
1	Project Planning	Scope, UI concept, project timeline
2	Requirements & System Design	README.md, requirements doc, wireframes, workflow diagram
3	Environment Setup	JDK, VS Code, Git, JFrame skeleton
4	GUI Layout & Theming	JTabbedPane, drop zones, styled buttons
5	Drag-and-Drop Integration	ImageTransferHandler, image preview, capacity label
6	LSB Embedding & Capacity	embedMessage(), calculateCapacity(), save PNG
7	LSB Extraction & Decoding	extractMessage(), ####END#### detection, auto-decode
8	Special Features & Polish	Clipboard copy, UI polish, error dialogs
9	Error Handling & Testing	Edge case handling, full bug fixes
10	Final Testing & Submission	All 10 test cases passed, final submission

5. Deliverables

- Final EasyStego.java — fully functional, tested, and uploaded to GitHub.
- 10 test cases executed — all passed successfully.
- Project report finalized and submitted.
- Application screenshots captured and included in documentation.
- All weekly workflow documents (Weeks 1–10) completed and submitted.
- GitHub repository finalized with complete commit history.

6. Tools & Resources

Development Environment:

- Java JDK 11+ — compiler and runtime
- Visual Studio Code with Java Extension Pack
- GitHub — version control and final file hosting

Testing Methods:

- Manual end-to-end testing of Hide and Reveal workflows
- Edge case testing for all error dialog scenarios
- Visual comparison of original and stego-images
- Cross-format testing with both PNG and JPEG input images